

Agent Runtime 技术全景

Agent Runtime 协议层技术调研

子领域：**Agent 间通信协议（Protocol）** 覆盖项目： projects/agent-runtime/protocol/A2A/ + projects/agent-runtime/protocol/modelcontextprotocol/

一、核心问题：Agent-to-Agent 与 Agent-to-Tool 通信标准化

协议层在 Agent Runtime 栈中要回答两个根本性问题：

问题	子问题	现状
Agent 如何调用工具？	工具发现、参数 schema、执行结果回传、权限控制、传输方式	MCP 已成事实标准
Agent 之间如何协作？	互相发现能力、协商任务、委托执行、状态同步、结果回传	A2A 成为共识方案
两个协议如何衔接？	Agent 通过 MCP 调用工具获得结果后，通过 A2A 转发给另一个 Agent？还是两个协议各自独立运作？	分层互补，非竞争
协议之上还缺什么？	治理、审计、跨 Agent 追踪、身份委派链	2026 年仍为开放问题

二、MCP vs A2A：分工与互补

2.1 一句话定位

- MCP (Model Context Protocol)**：Agent 的「手」——让 Agent 能操作外部世界（工具、数据、API）
- A2A (Agent-to-Agent Protocol)**：Agent 的「社交名片」——让 Agent 之间能互相发现、委托任务、交换结果

两个协议不是竞争关系，而是**分层互补**关系。类比 TCP/IP 体系：

治理层（权限 / 人机协同 / 审计 / 成本控制）	尚无统一标准
协同层 — A2A （Task 生命周期 · Agent Card · SSE 推送）	Agent ↔ Agent
发现层 — AGNTCY / A2A Agent Card	能力注册与查找
工具层 — MCP （Tools · Resources · Prompts · JSON-RPC）	Agent ↔ Tool / Data
传输层 — HTTP/2 · JSON-RPC · SSE · gRPC	

2.2 关键维度对比

维度	MCP	A2A
通信对象	Agent ↔ Tool / Data / API	Agent ↔ Agent
通信方向	垂直——Agent 向下访问系统	水平——Agent 之间对等协作
提出方	Anthropic (2024.11)	Google (2025.04)

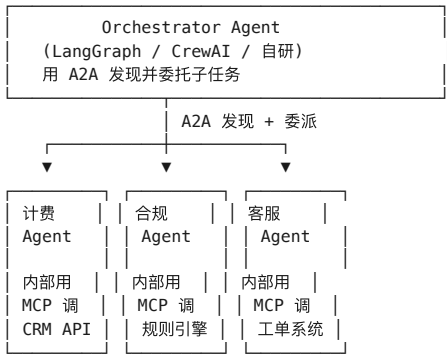
维度	MCP	A2A
治理	开源, Anthropic 主导 → Linux Foundation Agentic AI Foundation (2025.12)	Linux Foundation (2025 年中捐赠)
核心抽象	MCP Server 暴露 tools/resources/prompts	Agent Card 声明 Agent 能力
传输	JSON-RPC 2.0 over stdio (本地) 或 Streamable HTTP (远程)	gRPC/HTTP + SSE 流 + Webhook 推送
状态模型	Session-stateful (单次连接内)	Task-stateful (可跨数小时/天)
发现机制	会话内动态发现 (tools/list , resources/list)	Well-known URI (/.well-known/agent-card.json) + 注册中心
成熟度 (2026 中)	事实标准, 月 SDK 下载量 9700 万+	生产级, 150+ 组织支持
不适合	Agent 间协调、任务委派	Agent 调用文件系统/数据库等外部工具
Anthropic 示例	Claude Desktop 通过 MCP 调用文件系统、Slack、PostgreSQL	—
Google 示例	—	Cross-vendor agent orchestration
SDK	TypeScript v2 Beta, Python, Java, Kotlin, C#	Python, TypeScript, Java, Go, C#, Rust

2.3 分工是否会交叉？

答案是：存在模糊地带，但核心定位已区分。

- A2A 的 Task 内部，被调用 Agent 可能通过 MCP 调用工具完成任务——这是「A2A 承载 MCP 调用链」，不是协议冲突
- MCP 的 tools/call 理论上可以被另一个 Agent 调用——但 MCP 没有定义「哪个 Agent 有权调用哪个 Tool」「任务如何追踪到完成」——这正是 A2A 要补的
- 互补性 > 重叠性：MCP 解决 Agent 「用什么」，A2A 解决 Agent 「找谁」，两个维度正交

2.4 典型生产部署模式



三、协议层在 Agent Runtime 栈中的位置

Agent Runtime 栈的八个子层（按自下而上的逻辑排列）：

planner	规划与决策 — 「要做什么」
protocol	◀ 本调研聚焦 — 「怎么交流、怎么调用」
memory	上下文记忆 — 「记得什么」

tool	工具/能力 — 「有什么本事」
sandbox	代码执行 — 「在哪执行」
gateway	接入网关 — 「从哪里进来」
observability	可观测性 — 「跑得怎么样」
security	安全 — 「安不安全」

协议层是 **Runtime 的「连通性基础设施」**：它不决定 Agent 做什么（那是 planner），不提供工具实现（那是 tool / sandbox），但决定 Agent 之间、Agent 与工具之间**如何建立连接、传递语义、追踪状态**。它是 Agent 从单打独斗走向协作的第一道门槛。

协议层与相邻层的依赖关系

相邻层	对协议层的依赖	协议层对其的依赖
gateway	Gateway 需理解 MCP/A2A 协议字段才能做路由、限流、审计	—
observability	可观测性需要从 MCP/A2A 的请求/响应中提取 trace context	—
security	安全层需要验证 A2A Agent Card 签名、MCP OAuth token	—
planner	Planner 通过 A2A 发现可委托的 Agent，通过 MCP 发现可用的工具	—
memory	Memory 层可能需要通过 MCP 暴露为 Resource	—

四、A2A 协议深度：从 Agent Card 到 Task 生命周期

4.1 Agent Card——Agent 的「名片」

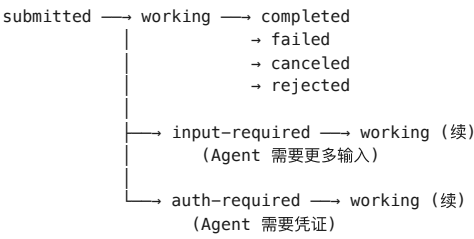
每个 A2A Agent 暴露 `/.well-known/agent-card.json`（RFC 8615），声明：

字段	含义
protocolVersion	A2A spec 版本号
name / description	Agent 人体可读的身份
url	JSON-RPC/HTTP 端点
capabilities	能力标志： <code>streaming</code> （SSE）、 <code>pushNotifications</code> （Webhook）
skills	技能列表： <code>id</code> 、 <code>name</code> 、 <code>description</code> 、 <code>tags</code> 、 <code>examples</code>
defaultInputModes / defaultOutputModes	支持的媒体类型
securitySchemes / security	声明的认证方式（OAuth 2.0、API Key、mTLS）
signatures	JWS 加密签名（可选），防止伪造 Agent Card
supportsAuthenticatedExtendedCard	是否支持认证后展示更详细的 Card

Agent Card 是 A2A 互操作的基础：主 Agent 从注册中心找到候选 Agent 的 Card 后，比较其 `skills` 是否匹配当前子任务，再发起 Task。

4.2 Task 状态机——长任务的生死周期

A2A 的 Task 是核心抽象，支持从数秒到数天的长任务：



关键设计： - **input-required 和 auth-required 不是终态**——它们是中断态，提供所需信息后任务继续 - **SSE 流事件**：TaskStatusUpdateEvent（状态转换）、TaskArtifactUpdateEvent（产出物更新，支持分块） - **Push Notification**：对超长任务（小时/天），Agent 主动 POST 状态变更到 Client 的 Webhook 端点 - **Resubscribe**：SSE 流意外断开后，Client 可用 tasks/subscribe 重建连接

4.3 A2A 服务接口（protobuf 定义）

RPC	说明
SendMessage	同步发送消息，返回完整 Task
SendStreamingMessage	发送消息并订阅 SSE 流
GetTask	查询 Task 当前状态（轮询模式）
ListTasks	列出历史/活跃 Task
CancelTask	取消运行中的 Task
SubscribeToTask	重新订阅 Task 的 SSE 流
GetExtendedAgentCard	获取认证后的扩展 Agent Card
CreateTaskPushNotificationConfig	配置 Webhook 推送
GetTaskPushNotificationConfig	查询推送配置
DeleteTaskPushNotificationConfig	删除推送配置

消息结构支持多模态 Part： text / file / data （通过 mimeType 区分）。

五、MCP 协议深度：三大原语与传输演进

5.1 核心抽象

原语	功能	示例
Tools	LLM 可调用的函数	查数据库、发邮件、调 API
Resources	结构化只读数据	读文件、查配置、获取文档
Prompts	预定义的可复用提示模板	“请总结以下代码:”

每个原语都有独立的发现端点： tools/list 、 resources/list 、 prompts/list 。

5.2 传输演进（关键变化）

阶段	传输方式	状态
2024.11 初版	stdio + HTTP SSE (legacy)	stdio 仍支持，SSE (legacy) 已废弃
2026.02 SEP-2243	Streamable HTTP + 标准化 HTTP 头	推荐方案
2026.07.28 (当前)	Streamable HTTP v2	移除 GET 流端点、移除协议级 session；Server→Client 请求改为 MRTR 内嵌

关键变化： - **移除协议级 session**：MCP 不再要求 long-lived session，每次请求互不依赖 - **MRTR (Multi Round-Trip Requests)**：Server 需向 Client 请求采样/确认时，不发起独立 JSON-RPC 调用，而是通过 `InputRequiredResult` 内嵌 - **Mcp-Method / Mcp-Name 标准 HTTP 头**：Gateway 层可直接根据 HTTP 头路由和限流，无需解析 JSON body

5.3 SDK 生态

- **TypeScript v2 Beta** (main branch)：按 2026.07.28 spec 实现，拆分为 `@modelcontextprotocol/server` + `@modelcontextprotocol/client`，工具/提示用 Standard Schema (Zod v4/Valibot/ArkType)，中间件支持 Express/Hono/Node
- **Python SDK**：稳定版，最广泛使用的参考实现
- **Java / Kotlin / C# / Go**：官方或社区维护

六、更广阔的协议版图：不只 MCP + A2A

协议	定位	状态
AGNTCY (Cisco)	Agent 发现与目录服务——「Agent 的 DNS」	75+ 公司，已捐 LF
ANP (Agent Network Protocol)	去中心化 Agent 通信 (DID + JSON-LD)	小众，跨组织场景
ACP (IBM, Agent Communication Protocol)	REST 多模态 Agent 消息	2025 年中并入 A2A
AGTP (IETF draft, 2026.03)	传输层统一协议，在各协议报文上加统一的身份/权限/预算头	早期草案
AP2 / UCP (Google)	Agent-to-Commerce 支付协议	领域特化，与 A2A/MCP 协作

趋势：协议从群雄割据走向 Linux Foundation 集中治理。2025 年 12 月，OpenAI、Anthropic、Google、Microsoft、AWS 等联合成立 **Agentic AI Foundation** (Linux Foundation 下属)，MCP 和 A2A 共同纳入中立治理。

七、趋势分析

7.1 确定性的趋势

1. **MCP 已成事实标准**：OpenAI Agents SDK、Google Gemini、Anthropic Claude、Cursor、Zed 全部原生支持 MCP。月 SDK 下载量 9700 万+。任何新 Agent 框架不集成 MCP 将难以获得生态采纳。
2. **A2A 成为 Agent 协作共识**：150+ 组织支持，Azure AI Foundry、AWS Bedrock、Google Cloud 全部集成。ACP 并入 A2A 消除了 REST 路线的分歧。
3. **协议分层已成共识**：MCP (工具层) → A2A (协同层) → Gateway (治理层) 的三层架构被广泛接受。
4. **Linux Foundation 统一治理**：消除了「选 Anthropic 还是 Google」的站队焦虑，降低了企业采纳门槛。

7.2 不确定性 / 开放问题

1. **治理层标准缺失**：MCP 和 A2A 都不回答「Agent 有权做什么」「跨 Agent 的权限如何委派」「什么场景必须有 Human-in-the-loop」。Gartner 预测到 2027 年底 40% 以上的 Agentic AI 项目会因治理不足而被取消。
2. **可观测性标准分裂**：OpenTelemetry GenAI、OpenInference、OpenLLMetry 各自定义了不同的 span kind 和 semantic convention。跨 MCP + A2A 调用链的端到端 trace 仍无统一方案。
3. **MCP Streamable HTTP 规范尚未完全稳定**：2026.07.28 的改动 (移除 session、移除 GET 流端点) 是 breaking change。依赖旧版 SSE 实现的 MCP Server 需要迁移。
4. **A2A 安全的深层问题**：签名 Agent Card 只验证身份，不解决「Agent B 接收任务后能否再委派给 Agent C」「委派链上的每一跳如何授权」。securitySchemes 字段只声明了认证方式，不定义委派语义。
5. **MCP + A2A 的组合模式尚未标准化**：一个 Orchestrator Agent 通过 A2A 委托任务 → 被委托 Agent 内部通过 MCP 调用工具 → 工具调用结果如何通过 A2A 回传给 Orchestrator —— 这个流程目前靠各框架自定，缺少统一的「嵌套调用追踪」规范。
6. **AGTP 能否成功?** IETF 的 AGTP 草案试图做传输层的「窄腰」(类比 TCP/IP)，但 MCP 和 A2A 的报文格式完全不同，统一头部格式对生态的价值取决于采纳率。

7.3 值得关注的信号

- **Gateway 层崛起**: AI Gateway（如 Portkey、Beam、Future AGI）正在成为「协议之上的新平台」，提供跨 MCP/A2A 的统一策略、审计和成本管理
- **Protocol-Neutral 的评估方案**: 最前沿的团队不再直接对 MCP 或 A2A 表面做评估，而是在一个 normalized trace 层做评估——无论协议是什么，trace 结构一致
- **Agent Card 从声明走向可验证**: Card 签名正在从「可选」走向「推荐」，下一步可能走向「必须」——这对企业采纳至关重要

八、项目全景表

项目	定位	提出方	治理	核心抽象	传输	成熟度	生态采纳	趋势
MCP (modelcontextprotocol/modelcontextprotocol)	Agent ↔ Tool / Data / API 标准化	Anthropic (2024.11)	LF Agentic AI Foundation	MCP Server (Tools/Resources/Prompts) + Client 发现-调用	JSON-RPC 2.0 over stdio / Streamable HTTP	事实标准	Anthropic Claude, OpenAI, Google, Microsoft, Cursor, Zed; 月 SDK下载 9700万+	主流生态
A2A (a2aproject/A2A)	Agent ↔ Agent 协作标准化	Google (2025.04)	Linux Foundation (2025 年中)	Agent Card（能力声明）+ Task（状态机）+ SSE/Push 通知	gRPC/HTTP + SSE + Webhook	生产级	150+ 组织, Azure AI, AWS Bedrock, Google Cloud 原生集成	生态萌芽

补充项目（调研范围内相关，但不属于 KG 当前收录）

项目	定位	与本领域的关系
AGNTCY (Cisco)	Agent 发现与目录	A2A Agent Card 的补充——解决「怎么找到 Agent」的问题
AGTP (IETF draft)	跨协议统一传输	试图在传输层解耦协议差异——对 MCP/A2A 互操作有潜在影响
ACP (IBM)	REST 多模态 Agent 消息协议	已并入 A2A，不再独立发展

九、关键结论

1. **协议层已收敛为 MCP + A2A 双支柱**，分别解决 Agent-to-Tool 和 Agent-to-Agent 通信，不竞争只互补。
2. **MCP 的技术路径清晰但仍在快速演进**: Streamable HTTP v2 (2026.07.28) 是 breaking change，移除 session 和 GET 流端点。依赖旧版 SSE 的 MCP Server 需要尽快迁移。
3. **A2A 的安全模型是其最薄弱的环节**: Agent Card 签名验证身份，但不定义委派链的权限传播。「Agent A 能否让 Agent B 代理其权限」这个问题在协议层面未解决。
4. **治理层缺失是 2026 年最大风险**: MCP + A2A 搭好了「怎么通」的基础设施，但「可不可以通」「通了之后谁负责」的标准空白，是 Gartner 预测 40% Agent 项目被取消的根因。
5. **对团队的建议**: 关注 A2A Agent Card 签名标准化进展、MCP Streamable HTTP 规范的最终冻结时间、以及 Linux Foundation Agentic AI Foundation 的治理层工作组成果。这三者的进展决定了 Agent 互操作生态的下一阶段走向。

本调研聚焦 projects/agent-runtime/protocol/ 下的 MCP 和 A2A 两个核心项目，结合 2025-2026 年行业进展和协议演进趋势完成。数据来源包括项目代码库分析（codebase-memory 对 A2A protobuf 规范的检索）、项目 summary.md、以及 2026 年中多份行业分析报告。